Department of Telecommunication and Networking

Specialized in cybersecurity

ESP32 Password Manager

Course : Cryptography

Year3 | Term 1

Instructed by : Mr Meas Sothearath

Date of submission : 20/12/2025

Submitted by : THY DAYUTH

Table of Contents

1. Introduction

1.1 Overview of the Project

This project implements an **ESP32-based password manager** that stores only salts and salted SHA-256 password hashes in Firebase Realtime Database. The ESP32 provides a small web UI (served on port 80) for creating named password entries and verifying them. The device uses the ESP32 hardware RNG for salt generation and mbedTLS SHA-256 for hashing.

1.2 Problem Statement

Storing plaintext passwords on embedded devices or in cloud backends is insecure. The project demonstrates a secure pattern: generate a per-entry random salt, compute SHA256(salt || password), and store only the salt and resulting hash. Verification recomputes the hash from user-supplied password and stored salt.

1.3 Motivation

- Teach secure handling of credentials on constrained devices (ESP32).
- Demonstrate use of cryptographic primitives (secure RNG, hashing) and cloud integration (Firebase).
- Provide a minimal, reproducible embedded example for learning secure storage and verification processes.

1.4 Related Cryptographic Concepts

- SHA-256 hashing (mbedTLS)
- Salting (per-entry random salt)
- Secure random generation (esp_fill_random)
- KDF / brute-force considerations (why single SHA-256 is not ideal in production)
- Data-in-transit vs at-rest considerations (HTTP vs HTTPS, cloud DB rules)

2. System Architecture

2.1 Overall Workflow

1. ESP32 boots, connects to WiFi, initializes Firebase, starts web server.
2. User visits ESP32 web UI in the same LAN and chooses Create or Verify.
3. **Create:** ESP32 generates random salt, computes SHA256(salt || password), writes {salt, hash} to Firebase at /passwords/<name>.
4. **Verify:** User supplies name, authHex (master gate), and password. ESP32 reads stored salt+hash, computes SHA256(salt || password) and compares. Returns VERIFIED/INVALID.

2.2 Key (Salt) Generation Process

- Function: genSaltHex(size_t lenBytes = 16)
- Uses esp_fill_random() to populate lenBytes of secure random bytes.
- Result encoded as lowercase hex (bytesToHex) and stored as salt in DB.
- Default salt length: 16 bytes (32 hex characters).

2.3 Password Hashing & Storage Process

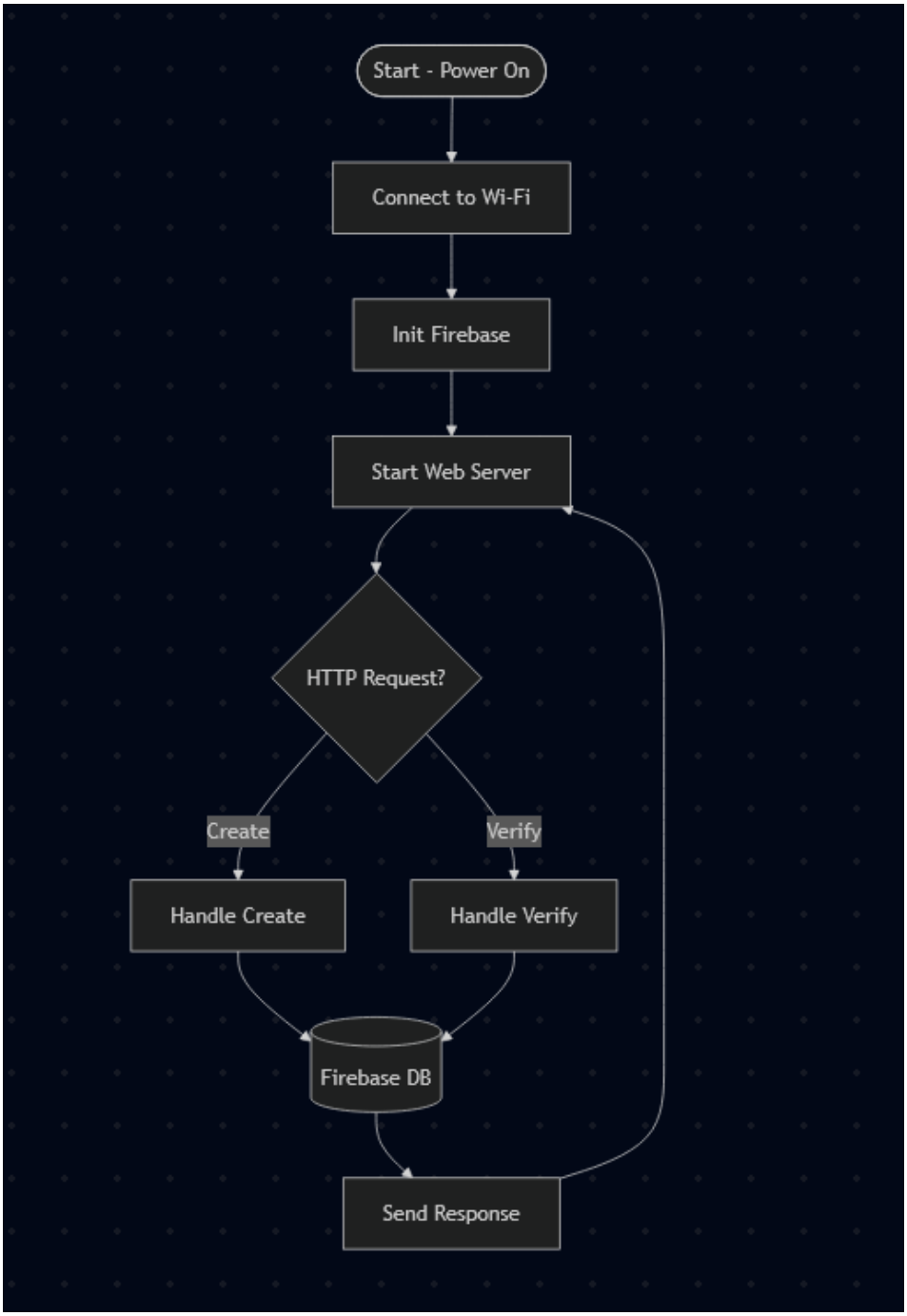
- Function: hashWithSaltHex(const String &saltHex, const String &password, String &outHashHex)
- Steps: decode salt hex → allocate buffer salt || password (salt bytes followed by raw password bytes) → compute SHA-256 over buffer → return hex hash.
- Storage path in Firebase: /passwords/<name>/salt and /passwords/<name>/hash.

2.4 Password Verification Process

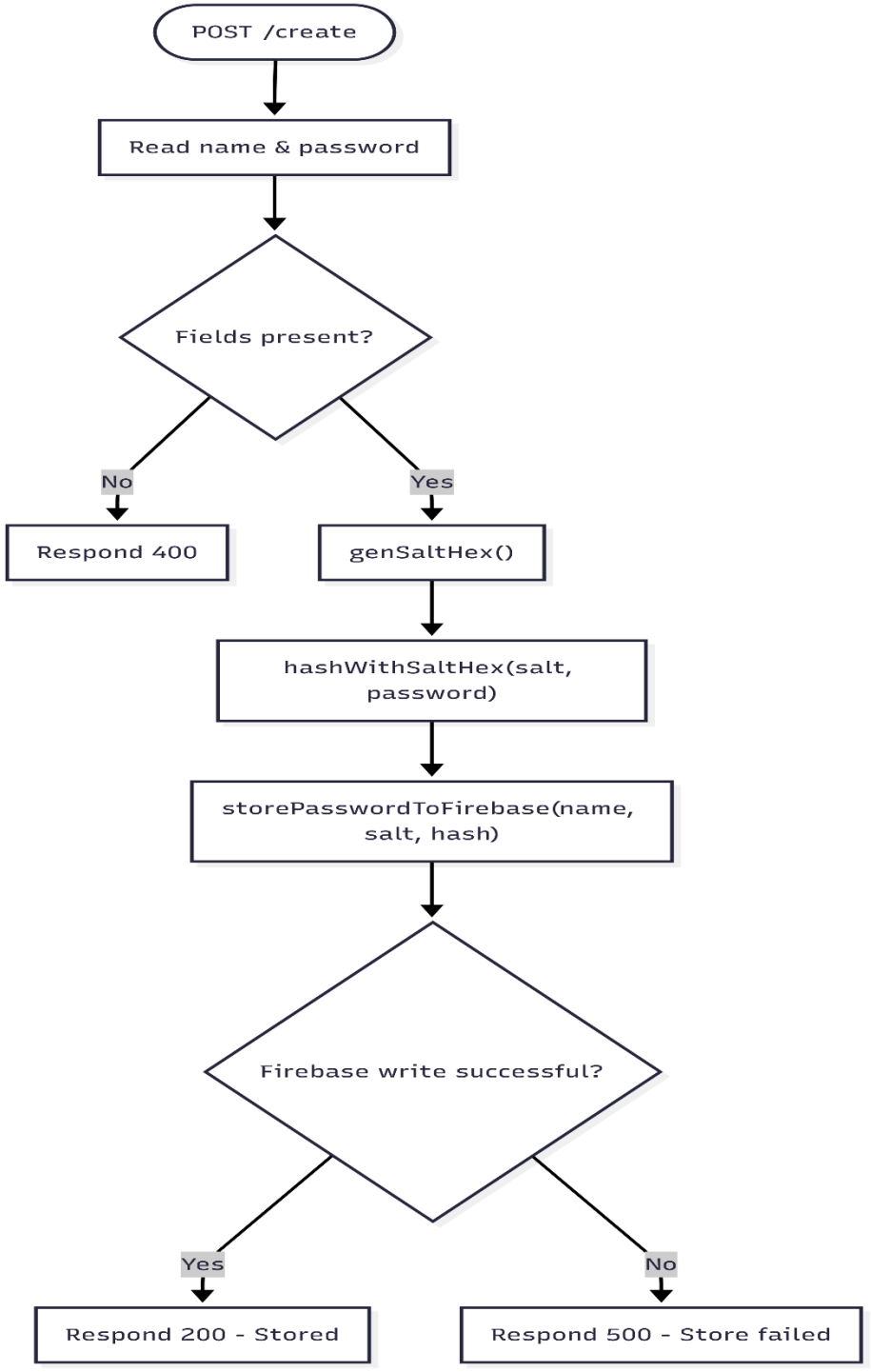
- Endpoint: /verify (POST)
- Gate: authHex compared against MASTER_PASSWORD_HASH (hard-coded). If mismatch → HTTP 403.
- Steps after auth: fetch salt+hash from Firebase, compute SHA256(salt || suppliedPassword), compare computed hash (case-insensitive) to stored hash → return VERIFIED or INVALID.

2.5 System Flowcharts & Diagrams

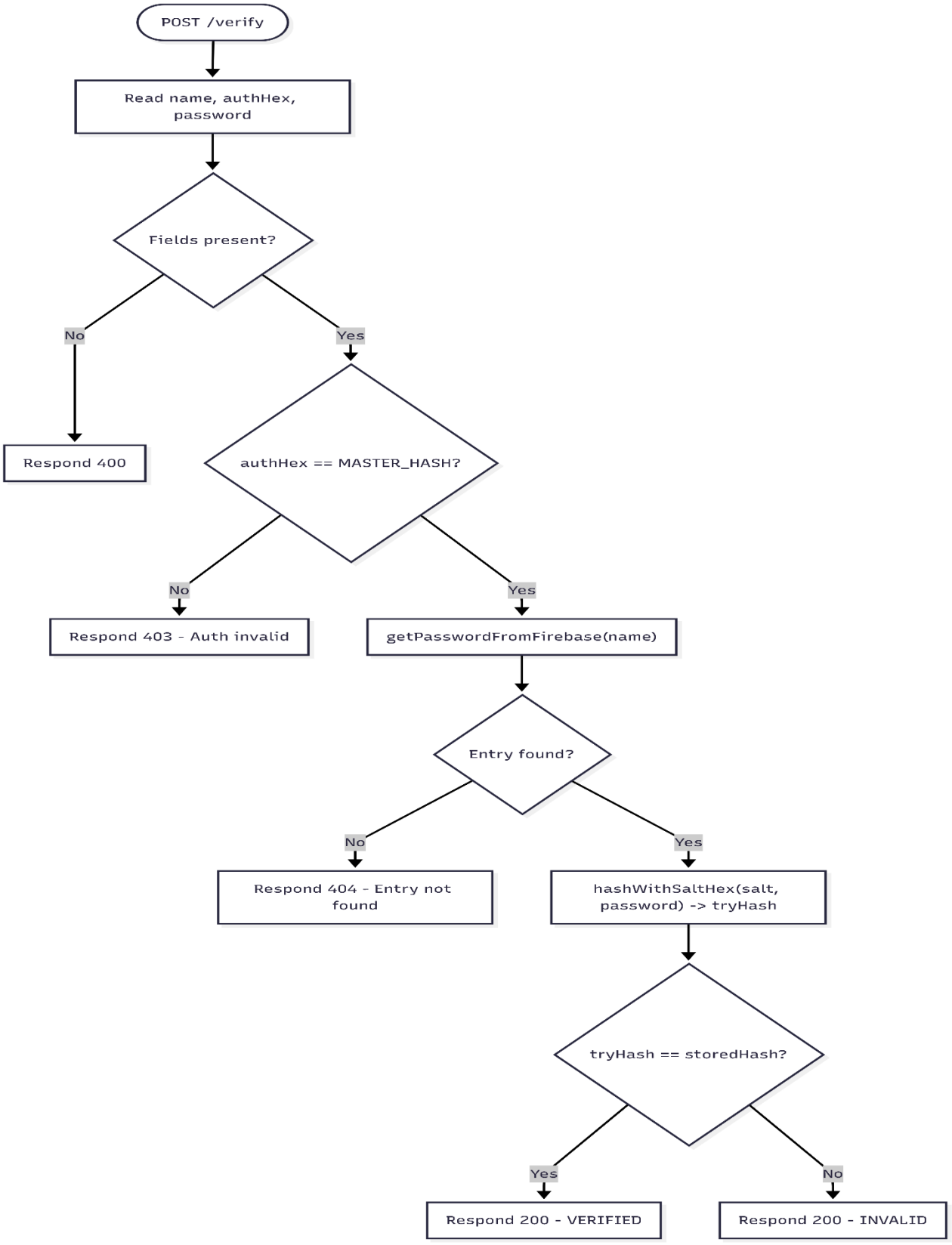
1. Overall System Flowchart



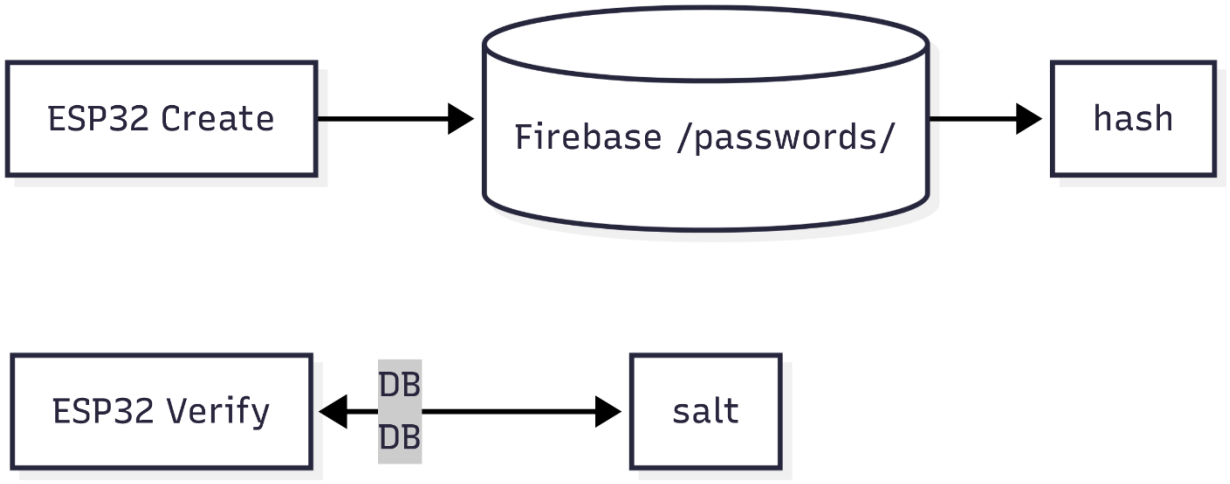
2. Create Password Flowchart



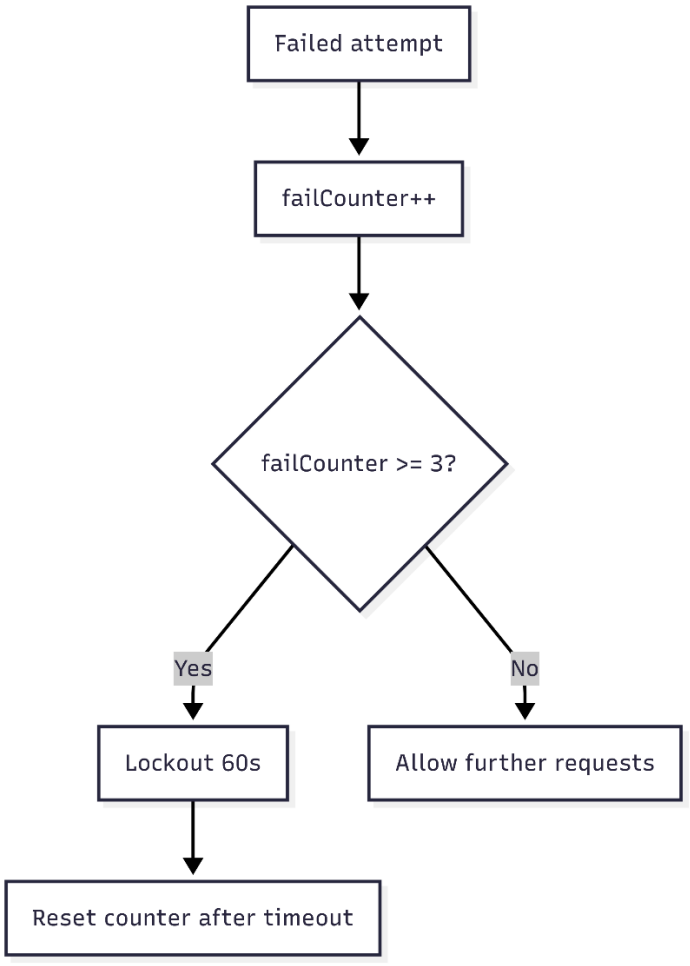
3. Verify Password Flowchart



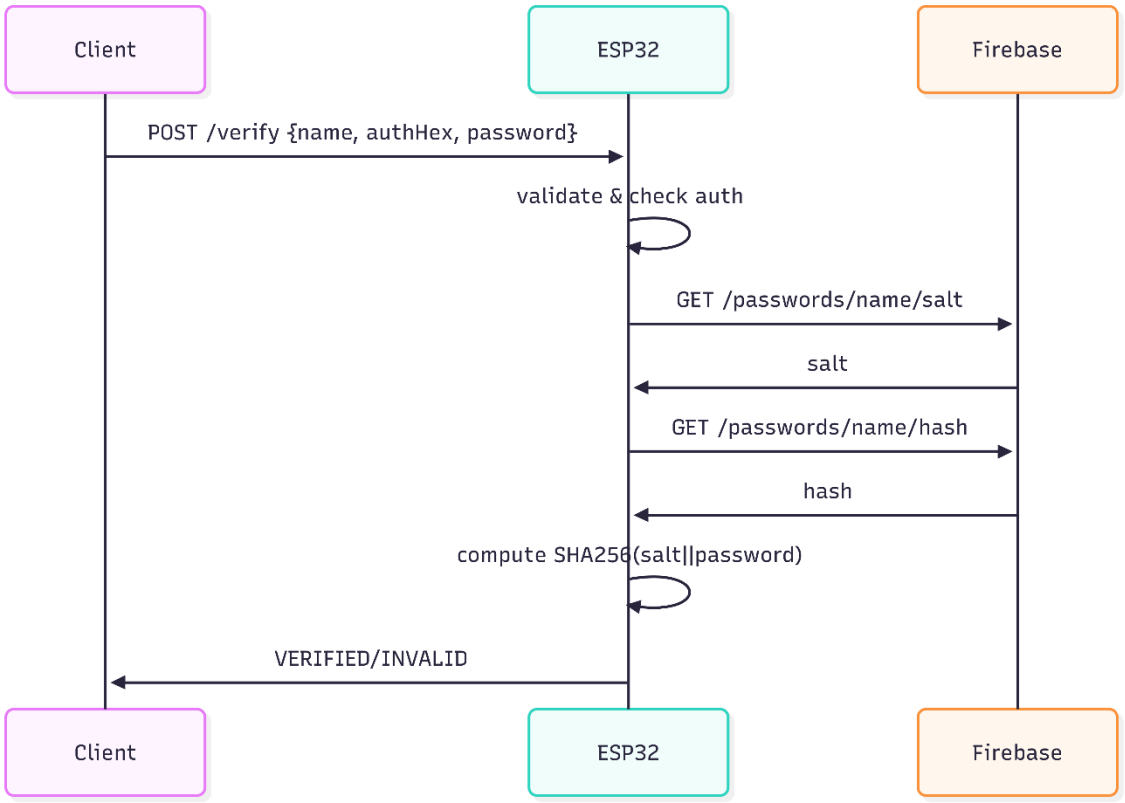
4. Firebase Data Flow



5. Error Handling & Security Flowchart



6. Sequence Diagram



3. Implementation Details

3.1 Technologies and Libraries Used

Component	Tech / Library
Language / Platform	Arduino C++ (ESP32 core)
Crypto	mbedTLS (mbedtls/sha256.h)
RNG	ESP32 esp_fill_random( )
Cloud DB	Firebase Realtime Database (Firebase_ESP_Client.h)
Web server	WebServer.h (Arduino)
Utilities	Standard C functions (malloc/memcpy/memset), Arduino String

3.2 Project Structure

```
SELF-PROJECT/
├─── README.md
├─── .gitignore
├─── docs/
│    └─── THY_DAYUTH_ESP32_PASSWORD_MANAGER.pdf
├─── data/
│    └─── sample.json
│    └─── structure.json
├─── src/
│    └─── main.ino
└─── IMG/…
```

3.3 Key Functions (from ESP32 sketch)

a. bytesToHex(const uint8_t *buf, size_t len)

- Converts bytes → hex string.

b. hexToBytes(const String &hex, uint8_t *out, size_t outLen)

- Converts hex string → byte array; validates length and characters.

c. sha256Hex(const String &input)

- Computes SHA-256 on input bytes and returns lowercase hex.

d. genSaltHex(size_t lenBytes = 16)

- Secure RNG to produce salt, returns hex.

e. hashWithSaltHex(const String &saltHex, const String &password, String &outHashHex)

- Core: decodes salt, concatenates salt||password, computes SHA-256, returns hex string. Zeroes sensitive buffers before free.

f. firebaseInit()

- Configures FirebaseConfig with API_KEY and DB_URL, calls Firebase.begin().

g. storePasswordToFirebase(const String &name, const String &saltHex, const String &hashHex)

- Writes two strings under /passwords/<name>/salt and /passwords/<name>/hash.

h. getPasswordFromFirebase(const String &name, String &outSaltHex, String &outHashHex)

- Reads salt and hash strings from DB; returns false on errors.

i. connectWiFi( )

- Connects to WiFi using SSID and PASSWORD, sets hostname, toggles LED_PIN on success.

j. Web handlers: handleCreatePost( ), handleVerifyPost( ), handleCreateGet( ), handleVerifyGet( ), handleQuit( )

- Serve static HTML pages and process POSTs.

3.4 Web UI Implementation

- Static HTML stored in PROGMEM strings: indexPage, createPage, verifyPage, quitPage.
- Forms post to /create and /verify.
- Minimalist UI intentionally avoids JS; server returns simple success/failure HTML.
- Note: pages served over HTTP (no TLS). Use only on trusted LAN or add gateway-authentication.

3.5 Security Practices Followed

Positive practices in code

- Uses hardware RNG (esp_fill_random) to generate salts.
- Zeroes buffers containing intermediate secrets before free (good hygiene).
- Salt + hash storage avoids plaintext password storage.

Weaknesses to address (explained later)

- Single SHA-256 is insufficient for resisting brute-force at scale — use PBKDF2 / Argon2 / scrypt.
- Hard-coded MASTER_PASSWORD_HASH and printing it to Serial is insecure.
- No Firebase authentication tokens or DB rules enforced in code.
- Web UI uses HTTP and lacks rate-limiting.

4. Usage Guide

4.1 Installation & Setup

1. Install Arduino IDE or PlatformIO and add ESP32 board support.
2. Install Firebase_ESP_Client library. (Library manager or PlatformIO library)
3. Configure constants at top of sketch:

```
#define LED_PIN    25
#define SSID       "YOUR_SSID"
#define PASSWORD   "YOUR_WIFI_PASSWORD"
#define HOSTNAME   "esp32-password"

#define API_KEY    "YOUR_FIREBASE_API_KEY"
#define DB_URL     "https://your-project-id.firebaseio.com/"

// replace or remove hard-coded master hash before committing
const char* MASTER_PASSWORD_HASH = "39b57...";
```

4. Do **not** commit real keys to a public repo. Use placeholders and document how to set runtime values.

4.2 Running the Program

- Upload sketch to ESP32.
- Open serial monitor at 115200 to observe boot logs and IP address.
- From a browser on same LAN, go to http://<esp_ip>/.

4.3 Creating (Storing) Password Entries

1. Visit /create.
2. Enter Entry Name (id) and Password.
3. Submit: ESP32 generates salt, computes hash, stores both to Firebase.
4. Expected feedback: "Stored successfully." or error message.

Firebase resulting JSON (example):

```
{
  "passwords": {
    "github": {
      "salt": "7f2a9c41d3b6e8f02a5b9c8a4d0f3a91",
      "hash":
"a1d4c6e2b7c98f13a9b87e4f5c2d1e8a3f0a6b4d2c9e8f1a7d6b5c4e3f2a"
    }
  }
}
```

4.4 Verifying Passwords

1. Visit /verify.
2. Enter Entry Name, authHex (master authorization), and Password to verify.
3. Submit: ESP32 reads salt+hash, recomputes SHA256(salt||password) and compares.
4. Outputs: "Password VERIFIED." or "Password INVALID." or HTTP errors for auth/entry-not-found.

4.5 Expected Outputs & Examples

- **Create example**: store entry alice with password p@ssw0rd → Firebase has salt and hash entries.
- **Verify valid**: verify alice with p@ssw0rd and correct authHex → VERIFIED.
- **Verify invalid password** → INVALID.
- **Bad authHex** → HTTP 403 "Authorization hash invalid".
- **Missing entry** → HTTP 404 "Entry not found".

5. Conclusion & Future Work

5.1 Summary

The ESP32 Password Manager demonstrates secure-sounding practices for embedded systems: per-entry salts, use of hardware RNG, hashing with SHA-256, and cloud-backed storage for salt+hash. It provides a compact learning platform for embedded cryptography and IoT-cloud integration.

5.2 Future work

Prioritized improvements before any production use:

- Replace single SHA-256 with a proper KDF (Argon2 or PBKDF2-HMAC-SHA256 with high iteration count).
- Remove hard-coded master secrets; use Firebase Authentication / server-side admin for authorization.
- Enforce Firebase Realtime Database security rules and require authenticated requests.
- Implement rate-limiting or lockout logic to mitigate brute-force attempts.
- Avoid serving sensitive endpoints over plain HTTP — add authentication gateway or TLS-capable proxy.
- Refactor into modules (crypto, firebase, web) and add unit tests.
- Document secret management and add README.md and SECURITY.md for the repository.

6. References

- mbed TLS documentation (mbedtls/sha256.h)
- Firebase Realtime Database: security rules & REST API docs
- NIST: Recommendations for key derivation and password storage (for PBKDF2 guidance)
- Arduino core for ESP32 documentation
- Best practice articles on KDFs: Argon2 / scrypt / PBKDF2